

Artificial Neural Networks: Dense

Machine Learning

Alexander Schoolcraft, Robbie Ward, Richard Zheng

October 16th, 2025

Model Design

Dense Artificial Neural Network (ANN-D) model is a fully connected neural network where each neuron within one layer is connected with each neuron in the next adjacent layer. By forming this network of connections, it has provided this model with the ability to capture complex feature relationships within user inputs, resulting in a simple yet robust design for recognizing handwritten digits.

This model consists of 4 different layers, including an input layer, two hidden layers and the output layer. The input layer receives 784-dimensional feature vector from the 28x28 pixel values of each MNIST image. The two hidden layers are made up of 256 and 128 neurons respectively. Both layers have applied ReLU (Rectified Linear Unit) with a 10% dropout rate that randomly deactivates a portion of the neurons to improve the model's generalization.

The output layer is made up of 10 neurons, corresponding to each one of the digit classes from 0 to 9. This layer allows the model to convert raw outputs into class probabilities by measuring the total loss using categorical cross entropy and SoftMax activation, which includes complex calculation including accuracy, confidence, and weights. These steps effectively help with guiding the learning process during training.

Hyperparameter Tuning

Hyperparameters in dense neural networks include values like learning and dropout rates, the. In this neural network, there are a total of 7 hyperparameters that require tuning. The first hyperparameter is its learning rate. Since this model was trained using the Adam optimizer, a learning rate that is too high can cause unstable training and prevent convergence. Moreover, a

learning rate that is too low can unnecessarily slow down the training speed. Therefore, to train the model with the MNIST dataset, maintaining a moderate learning rate can provide smooth convergence across the 20 training epochs.

The second hyperparameter in this model is the number of epochs. ANN was trained for 20 epochs. As the model learns during training, the loss values often decrease steadily over epochs. At Epoch 17, the model's loss value began to slowly fluctuate. This is a sign that the model is slowly overfitting. A good strategy to mitigate this hyperparameter is to stop training around Epoch 17 or to apply stronger regularization.

The third hyperparameters lie in the two hidden layers with 256 and 128 neurons per layer respectively. These two layers provided enough neurons to learn from the MNIST dataset without the model experiencing severe overfitting. When there is an abundance of neurons present in the model, noise or irrelevant patterns might be memorized rather than the actual signal that is important for the model. Therefore, adjusting these two layer's sizes is a way to control the complexity of the model.

The fourth hyperparameter is the model's Activation Function. This function defines how an output from a neuron is transformed before being passed on to the next layer. This function plays an important role in the definition of how this ANN learns. For this model, Rectified Linear Unit (ReLU) is being used to define the output. Without this function, this entire model will simply behave like a single big linear layer without the ability to learn complex patterns.

$$f(x) = \max(0, x)$$

Figure 1: Rectified Line Unit Calculation

The fifth hyperparameter in this model is its dropout rate applied to each hidden layer to avoid overfitting issues. During training, 10% of the neuron were randomly selected to be “turned off” each forward pass. This dropout rate is important to the model as it forces the network to learn more robust representation. It also allowed regularization within the model without significantly harming the training accuracy

The Adam Optimizer is the model’s sixth hyperparameter. This optimizer is used to help facilitate the learning rate for each of the parameters during training. Adam Optimizer is known for its ability to keep a running average of all past gradients for smoother updates, as well as its ability to scale the learning up or down based on recent gradient. This is to ensure that updates can be done without issues. By leveraging these two advantages from two other optimizers, Adam Optimizer can adjust the speed at which it adjusts its weight and bias.

The final hyperparameter in this network is the Step Learning Rate (Step LR) scheduler. This scheduler’s main purpose is to fine tune the model and reduce the learning rate near convergence. This way, it prevents the optimizer from taking steps that are too large. Additionally, it also helps mitigate the issue of overfitting when training loss stops improving.

Results

This dense ANN was training using the MNIST dataset made up of 70000 handwritten digits. Out of these 70000 images, 50000 of them were used for training. The remaining 20000 images were equally split between validation and testing. The performance of this model is being evaluated using simple metrics, including its accuracy, loss, precision, recall, and F1-score. A visual analysis is also assessed by analyzing training curves, confusion matrix, and sample predictions.

The accuracy over Epochs curves graphically shown the accuracy trend of the model for the duration of 20 training epochs. Initially, the accuracy of the model was at 90.43% at the beginning of training. As the model continuously acquired knowledge through training, their accuracy quickly rose to 99.65% by the end of training (epoch 20). Both the training and validation curve trends upward during the first 10 epochs. However, after Epoch 10, the validation accuracy curve plateaued while the training accuracy curve continued to rise incrementally (Figure 1). This is a good sign that the model has successfully being trained on the discriminative features through the MNIST dataset. Minor overfitting can be seen from the small gap between training and validation curve at epoch 20. This is common in all dense neural networks as it begins to memorize the examples from training instead of generalizing fully. With the differences maintained below 2%, this model has demonstrated its strong capacity to generalize examples.

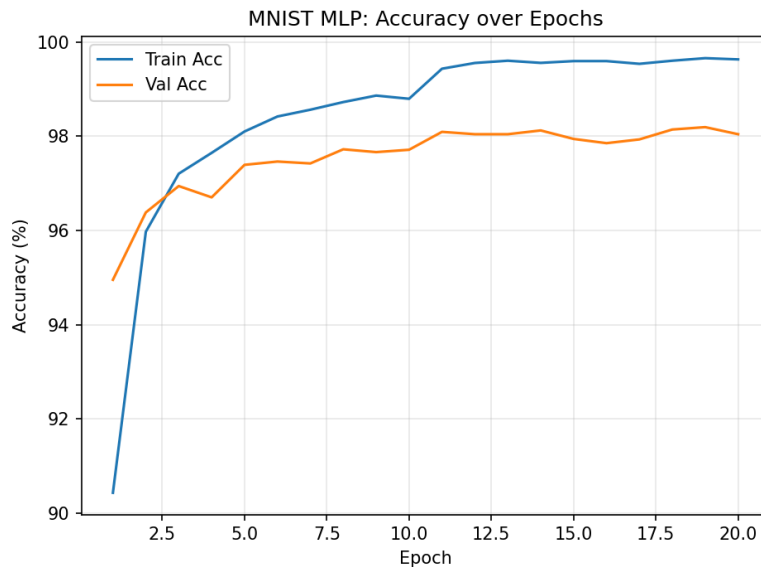


Figure 1: Training and Validation Accuracy Curve

As the model continues to learn from the MNIST dataset, the training and validation loss have experienced a sharp decline over the 20-epoch span. When training was first started, the model experienced a training loss of 0.3182 and a validation loss of 0.1606. This relatively huge training loss is a good sign in this case, as it indicates that the neural network is still learning the different handwriting features. By Epoch 3, these losses in training and validation has dropped to 0.0710 and 0.0999 respectively. As the model continues learning, these losses continue to decline, but at a much slower rate. Between Epochs 5 and 10, training loss only fell from 0.0602 to 0.0329, while validation loss only fell from 0.0859 to 0.0715. This is a clear sign that this model has transitioned from general feature extraction to detailed pattern recognition. The fine-tune recognition process continues until Epoch 15, during which the validation loss fluctuates between 0.0713 and 0.0754 periodically. By the end of training, the training loss reached 0.0118 and the validation loss reached 0.0789 (Figure 2). The gap between these two curves at this point is relatively small, representing a stable and low trend and a successful refining process using the StepLR scheduler. The overall rapid convergence is driven by the Adam optimizer's adaptive learning rate mechanism, which is designed to minimize the loss function during training.

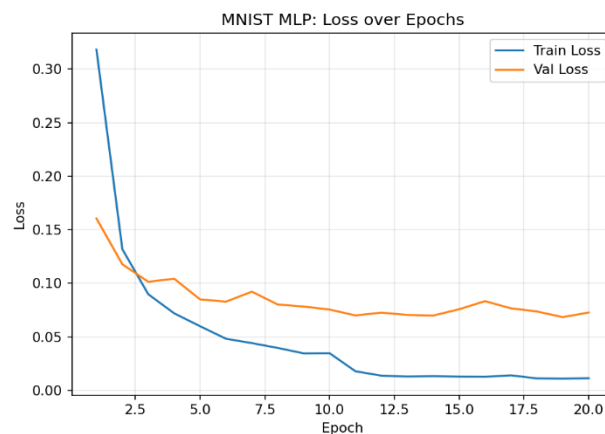


Figure 2: ANN-D Loss curve

To put this model to test, 24 out of the 20000 handwritten digits were randomly selected to test the model's recognition accuracy. These 20000 images were previously set aside while the model was trained on the remaining 50000 images in the dataset. For evaluating the accuracy of the model, 24 random samples were selected to test the model. Each panel has the handwritten digits, the actual number that the user wants to write out, and the model's predicted number with the confidence score (Figure 3). From this high accuracy rate from the model, it is safe to say that this model can classify a wide range of handwritten numbers correctly, regardless of the variation in the user's stroke widths, rotation, or handwriting styles.



Figure 3: Random samples with prediction, actual, and confidence

Although this model has a decent accuracy on recognizing users' handwriting, there are still cases where the model misclassified the number that user wrote. Based on this implementation, when a user wrote a number that resembles two other different numbers, the model cannot distinguish what the actual number the user intended to write. For example, a loosely written number "9" can be mistaken for the number "1" and "4" (Figure 4). This is one of the instances a model is confident about their prediction but assigned the wrong label. From the programming standpoint, the result implies that these ambiguous features on a handwritten digit

are closely overlapping in the learned feature space. Overall, these classification errors mostly stemmed from vague data rather than inadequately trained models.

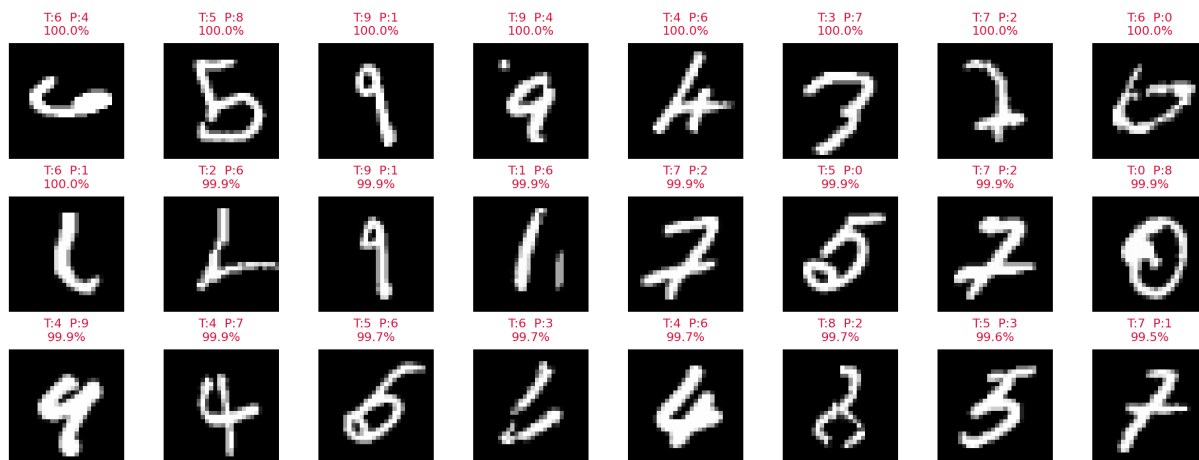


Figure 4: Examples of incorrectly classified digits and model's confidence levels

These two result graphs exhibit the quality of handwritten digit recognition with this dense neural network. From the correctly identified samples, we can confirm that this model has the ability to generalize learned features. As this neural network is only learning the generalized feature of each digit, this model still lacks the ability to distinguish irregularities in handwriting. However, the visual analysis reinforces quantitative findings of which the model delivers a high accuracy, with only a small marginal degradation when it comes to visually ambiguous digits.

Discussion & Future Work

This ANN-D model works perfectly when being used to recognizing handwritten digits from the MNIST dataset. This model is able to recognize 98.22% of the handwritten digits correctly from looking at its precision, recall, and F1-scores for all of the ten-digit classes. This model further proved that through appropriate hyperparameter tuning and regularization, even the simplest fully connected neural network architecture can achieve strong generalization on a benchmark dataset.

Using these hyperparameter, this neural network shows a smooth learning process, reaching a high training accuracy of 99.65% by Epoch 20. The Adam optimizer and the StepLR scheduler helped refining the learning rate dynamically. The pre-defined 10% dropout rate ensures the model does not learn any irrelevant data from the dataset. The confusion matrix and handwriting prediction samples further confirm the model's ability to accurately recognize handwritten digits, with the most common misclassification when recognizing a poorly written 4, 9, 5, and 6.

This model highlights the efficiency and effectiveness of incorporating neural networks with image recognition and classification. Future work on this model can be focused on incorporating data augmentation and early stopping to enhance the feature extraction process and push the accuracy beyond 99%.